

# Lumen (Agente 04 · Copilot)

Cómo funciona el agente, explicado desde cero — esquema de trabajo, esquema de la base de datos y glosario.

Proyecto Ágora · Mitumi · Documento de referencia para personas que empiezan · Actualizado: 10/07/2026

## Para quién es este documento

Está escrito para alguien que **no ha programado** este proyecto (o no ha programado nunca). No hace falta saber nada previo: cada término técnico se explica la primera vez que aparece, y al final hay un glosario. Si solo quieres una cosa: **las consultas a la base de datos están en el archivo `integrations/db_backend.py`** (lo explicamos en la sección 6).

## Contenido

1. [Qué es Lumen, en una frase](#)
2. [Las 6 palabras que conviene entender primero](#)
3. [El recorrido de una pregunta \(diagrama\)](#)
4. [La regla de oro: el LLM nunca toca la base de datos](#)
5. [Mapa de archivos: qué hace cada uno](#)
6. [Dónde están las consultas a la base de datos](#)
7. [El esquema de la base de datos \(tablas y campos\)](#)
8. [Configuración y seguridad](#)
9. [Qué pasa cuando algo falla](#)
10. [Glosario completo](#)

## 1 · Qué es Lumen, en una frase

Lumen es un **asistente que responde preguntas en lenguaje natural sobre los datos de la plataforma Ágora** (eventos, clientes, presupuestos, ponentes, salas...), **leyéndolos de una base de datos, sin cambiar nunca nada**.

La analogía más útil: imagina un **bibliotecario** al que le preguntas de viva voz, mira sus fichas y te contesta. No reescribe los libros, no tira nada, no firma nada por ti. Solo *consulta* y *explica*. Lumen es exactamente eso: un agente **de solo lectura**.

### ✓ Lo que Lumen SÍ hace

Entender tu pregunta escrita, buscar el dato en la base de datos y devolvértelo redactado en español claro, diciéndote de qué tabla salió si lo pides.

### ✗ Lo que Lumen NUNCA hace

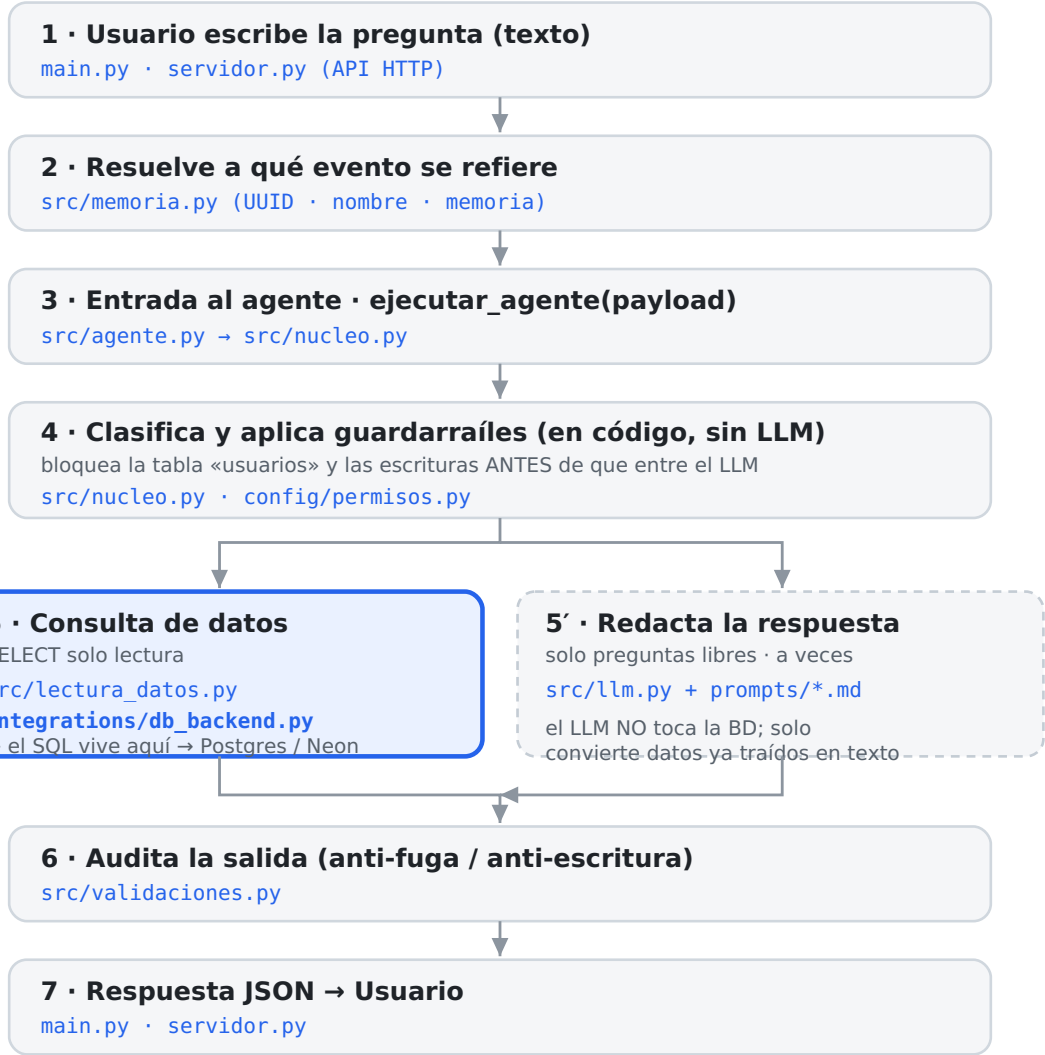
Modificar, borrar, aprobar o crear datos; enviar correos o mensajes; tocar la tabla de `usuarios` o contraseñas. Estas prohibiciones están escritas en el código, no son solo una buena intención.

## 2 · Las 6 palabras que conviene entender primero

| Palabra           | Qué significa aquí (en simple)                                                                                                                                                                             |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Base de datos     | Un almacén ordenado de información, como un conjunto de hojas de cálculo (aquí se llaman <i>tablas</i> ): una de eventos, otra de clientes, otra de presupuestos...                                        |
| Postgres / Neon   | Postgres es el <i>tipo</i> de base de datos que usamos. Neon es la empresa que la aloja en internet. Juntos: "la base de datos real, en la nube".                                                          |
| LLM               | "Modelo de lenguaje": la inteligencia artificial que entiende y redacta texto (aquí, el modelo <i>Llama 3.3</i> servido por <i>Groq</i> ). Sirve para <i>escribir la respuesta</i> , no para buscar datos. |
| API               | Una "puerta" por la que un programa habla con otro. Lumen tiene una API (el archivo <code>servidor.py</code> ) para que una página web pueda preguntarle.                                                  |
| <code>.env</code> | Un archivo de configuración con los <i>secretos</i> (la clave del LLM y la contraseña de la base de datos). No se sube a internet.                                                                         |
| Consulta / SQL    | Una "pregunta" formal a la base de datos, escrita en un lenguaje llamado SQL (p. ej. <code>SELECT * FROM eventos</code> = "dame todos los eventos"). La escribe el código, no el LLM.                      |

## 3 · El recorrido de una pregunta (diagrama)

Cuando escribes una pregunta, atraviesa estos pasos. En cada caja se indica **qué archivo** se encarga de ese paso.



 Borde azul = ruta hasta la base de datos: aquí están las consultas SQL.

## El mismo recorrido, paso a paso

| Paso | Qué ocurre                                                      | Archivo                                                                      | Nota para principiante                                                                                                                           |
|------|-----------------------------------------------------------------|------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | Escribes tu pregunta en texto.                                  | <code>main.py</code><br><code>servidor.py</code>                             | <code>main.py</code> es para probar desde la consola (terminal). <code>servidor.py</code> es la API para que una web te hable.                   |
| 2    | Se decide <i>de qué evento</i> hablas.                          | <code>src/memoria.py</code>                                                  | Como nadie escribe códigos largos a mano, entiende el <i>nombre</i> del evento y recuerda el último del que hablabas ("¿y su presupuesto?").     |
| 3    | Se entra al "cerebro" del agente.                               | <code>src/agente.py</code><br>→ <code>src/nucleo.py</code>                   | <code>agente.py</code> es solo la puerta oficial de entrada; la lógica de verdad vive en <code>nucleo.py</code> .                                |
| 4    | Se clasifica la pregunta y se aplican las reglas de seguridad.  | <code>src/nucleo.py</code><br><code>config/permisos.py</code>                | Aquí se <b>bloquea</b> cualquier intento de tocar <code>usuarios</code> o de escribir, <i>antes</i> de que la IA participe.                      |
| 5    | Se consultan los datos en la base de datos (solo lectura).      | <code>src/lectura_datos.py</code><br><code>integrations/db_backend.py</code> | <b>Aquí están las consultas SQL.</b><br><code>lectura_datos.py</code> decide qué pedir; <code>db_backend.py</code> lo pide de verdad a Postgres. |
| 5'   | <i>Solo a veces:</i> la IA redacta la respuesta con esos datos. | <code>src/llm.py</code><br><code>prompts/*.md</code>                         | Muchas preguntas (conteos, listados) ni usan la IA. Y si la IA falla, hay una respuesta de reserva automática.                                   |
| 6    | Se revisa la respuesta antes de entregarla.                     | <code>src/validaciones.py</code>                                             | Última barrera: aunque la IA se equivocara, aquí se borra cualquier fuga sobre <code>usuarios</code> o cualquier acción de escritura.            |
| 7    | Se devuelve la respuesta y la lees.                             | <code>main.py</code><br><code>servidor.py</code>                             | La respuesta viaja como <i>JSON</i> (un formato de datos); tú solo ves el texto del campo <code>resumen</code> .                                 |

## 4 · La regla de oro: el LLM nunca toca la base de datos

Es el punto más importante y el que más se malentiende. Mucha gente imagina que la IA "traduce tu pregunta a una consulta y se la manda a la base de datos". **Aquí no es así.**

- Quien decide qué consultar y quien escribe el SQL es **código normal** (`nucleo.py` + `lectura_datos.py` + `db_backend.py`), no la IA.
- La IA (`llm.py`) entra **al final**, y solo para redactar en español unos datos que el código *ya trajo y ya filtró*.
- La IA nunca ve la tabla `usuarios`: se excluye en el código antes de darle nada.

### Por qué se diseñó así

Por seguridad. Aunque alguien intentara "engañar" a la IA con una pregunta tramposa, o la IA se inventara algo, **no puede saltarse los permisos de datos porque nunca habla con la base de datos**. La IA es un "redactor", no un "consultor". Esa separación es la que hace que el agente sea fiable.

## 5 · Mapa de archivos: qué hace cada uno

Agrupados por su papel en el recorrido. Los "enlaces" entre ellos son las líneas `import` del código (si quieres, puedes buscarlas).

### • Entrada (por dónde llega tu pregunta)

`main.py` — chat por consola y modo `--demo`. `servidor.py` — API web (Flask) para el frontend.

### • Contrato de entrada

`src/agente.py` — la función oficial `ejecutar_agente(payload)`. No tiene lógica: solo reexporta desde `nucleo.py`.

### • Cerebro (decide todo)

`src/nucleo.py` — clasifica la pregunta, aplica bloqueos y orquesta el resto. `src/memoria.py` — recuerda el evento del que hablas.

### • Datos (habla con la base de datos)

`src/lectura_datos.py` — decide qué leer y comprueba permisos. `integrations/db_backend.py` — el SQL y la conexión a Postgres.

### • Inteligencia artificial (redacta)

`src/llm.py` — cliente del modelo (Groq). `src/prompts.py` + `prompts/*.md` — las instrucciones que se le dan a la IA.

### • Seguridad y contrato de salida

`src/validaciones.py` — audita la respuesta final. `config/permisos.py` — permisos fijos (nunca escribir, tabla `usuarios` prohibida). `src/schemas.py` — comprueba la forma de la entrada/salida.

### • Configuración

`config/settings.py` — lee el archivo `.env`. `.env` — los secretos reales (clave de Groq, `DATABASE_URL`).

### • Utilidad de comprobación

`integrations/verificar_conexion_bd.py` — script que comprueba que la base de datos real coincide con el esquema documentado.

## 6 · Dónde están las consultas a la base de datos

### Respuesta directa

Las consultas SQL están en `integrations/db_backend.py`. Es el **único** archivo que abre la conexión a Postgres y escribe `SELECT`. Todo lo demás pasa por él para leer datos.

Hay dos niveles, y conviene distinguirlos:

| Archivo                                 | Qué contiene                                                                                                                                                                                                                                                                                                                                   |
|-----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>integrations/db_backend.py</code> | El SQL real y la conexión. Dos funciones: <code>obtener_por_id(tabla, id)</code> → <code>SELECT * FROM &lt;tabla&gt; WHERE id = ...</code> , y <code>listar(tabla)</code> → <code>SELECT * FROM &lt;tabla&gt;</code> . La conexión se abre en <b>modo solo lectura</b> , así que Postgres rechaza cualquier escritura aunque hubiera un error. |
| <code>src/lectura_datos.py</code>       | La capa de arriba: funciones con nombre de negocio ( <code>resumen_evento</code> , <code>eventos_por_estado</code> , <code>contexto_completo_evento</code> ...) que deciden <i>qué</i> pedir y comprueban los permisos (bloquea <code>usuarios</code> ) antes de llamar a <code>db_backend.py</code> .                                         |

### Detalle importante

El nombre de la tabla en el SQL **nunca** viene de lo que tú escribas: sale siempre de una lista fija de tablas permitidas en `config/permisos.py`. Los valores (identificadores, filtros) van "parametrizados", que es la forma segura de evitar el ataque clásico de *inyección SQL*.

## 7 · El esquema de la base de datos (tablas y campos)

La base de datos tiene 8 tablas de negocio que Lumen puede leer, más 1 tabla (`usuarios`) totalmente prohibida. En cada tabla, el identificador único siempre se llama `id`. Los campos que empiezan por `id_` son "enlaces" a otra tabla (claves foráneas).

## eventos — la tabla central

| Campo                                              | Qué es                                                                    |
|----------------------------------------------------|---------------------------------------------------------------------------|
| <code>id</code>                                    | Identificador único del evento (un código tipo UUID).                     |
| <code>nombre_evento</code>                         | Nombre del evento (lo que la gente escribe al preguntar).                 |
| <code>ciudad</code>                                | Ciudad donde se celebra.                                                  |
| <code>lugar_confirmado</code>                      | Si el lugar está confirmado o no.                                         |
| <code>fecha_inicio</code> · <code>fecha_fin</code> | Fechas de inicio y fin.                                                   |
| <code>numero_personas</code>                       | Asistentes previstos.                                                     |
| <code>tipo_evento</code> · <code>nota</code>       | Tipo de evento y una nota libre.                                          |
| <code>id_presupuesto</code>                        | Enlace → tabla <code>presupuestos</code> .                                |
| <code>id_cliente</code>                            | Enlace → tabla <code>clientes</code> .                                    |
| <code>id_estado</code>                             | Enlace → tabla <code>estados</code> (borrador, confirmado, cancelado...). |
| <code>id_sala</code>                               | Enlace → tabla <code>salas</code> .                                       |
| <code>id_ponencia</code>                           | Enlace → tabla <code>ponencias</code> (y por ella, al ponente).           |

## Las otras tablas de negocio

| Tabla        | Para qué sirve · campos principales                                                                                                                                                                                                                                                             |
|--------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| clientes     | Quién encarga el evento. <code>cliente</code> , <code>email</code> , <code>telefono</code> , <code>empresa</code> , <code>sector</code> , <code>ciudad</code> .                                                                                                                                 |
| presupuestos | El coste del evento, desglosado. <code>total</code> , <code>estado_presupuesto</code> , <code>fecha</code> , y bloques de <code>ubicacion</code> , <code>catering</code> , <code>audiovisuales</code> , <code>otros</code> (cada uno con su nota y su precio), más <code>observaciones</code> . |
| salas        | La sala concreta. <code>nombre_sala</code> , <code>tipo_sala</code> , <code>capacidad_max_sala</code> , <code>nota_sala</code> , <code>id_espacio</code> (→ espacio que la contiene).                                                                                                           |
| espacios     | El edificio/recinto donde están las salas. <code>nombre_espacio</code> , <code>ciudad</code> , <code>direccion</code> , <code>aforo</code> , datos de contacto.                                                                                                                                 |
| ponencias    | La charla y la logística del ponente. <code>horario_ponencia</code> , <code>presentacion_link</code> , <code>billete_ida_link</code> , <code>billete_vuelta_link</code> , hotel y transporte, <code>id_ponente</code> .                                                                         |
| ponentes     | La persona que da la charla. <code>nombre_ponente</code> , <code>email</code> , <code>telefono</code> , <code>empresa</code> , <code>cargo</code> , enlaces a foto/CV.                                                                                                                          |
| estados      | La lista de estados posibles de un evento. Solo <code>id</code> y <code>descripcion</code> (p. ej. "Borrador", "Confirmado", "Cancelado").                                                                                                                                                      |

## Cómo se relacionan (quién apunta a quién)

| Relación                                                           | En cristiano                                             |
|--------------------------------------------------------------------|----------------------------------------------------------|
| <code>eventos.id_cliente</code> → <code>clientes.id</code>         | Cada evento tiene un cliente.                            |
| <code>eventos.id_estado</code> → <code>estados.id</code>           | Cada evento tiene un estado.                             |
| <code>eventos.id_sala</code> → <code>salas.id</code>               | Cada evento tiene una sala...                            |
| <code>salas.id_espacio</code> → <code>espacios.id</code>           | ...y cada sala está dentro de un espacio.                |
| <code>eventos.id_presupuesto</code> → <code>presupuestos.id</code> | Cada evento tiene un presupuesto.                        |
| <code>eventos.id_ponencia</code> → <code>ponencias.id</code>       | Cada evento enlaza como mucho con <b>una</b> ponencia... |
| <code>ponencias.id_ponente</code> → <code>ponentes.id</code>       | ...y cada ponencia, con <b>un</b> ponente.               |

### Un matiz que confunde

En este modelo **un evento tiene como mucho un ponente** (a través de su ponencia). Si alguien pregunta por "los ponentes" en plural, Lumen responde según lo que exista de verdad: cero o uno, sin suponer que puede haber varios.



La tabla prohibida: **usuarios**

Exclusión dura, no negociable

La tabla `usuarios` (cuentas y accesos de la plataforma) está **totalmente fuera del alcance** de Lumen. Nunca la consulta, nunca revela ni confirma nada sobre contraseñas o accesos. Esto está reforzado en *tres sitios*: las instrucciones de la IA (`prompts/prompt_sistema.md`), el código de datos (`lectura_datos.py` / `db_backend.py`) y la auditoría final (`validaciones.py`). Si preguntas por ella, Lumen lo marca como bloqueo de riesgo "alto".

Datos personales

Campos como el email o el teléfono de ponentes y clientes son datos personales. Lumen puede consultarlos para uso interno del equipo, pero no genera listados masivos exportables salvo que se le pida de forma acotada, y marca como "requiere validación humana" cualquier intento de sacarlos fuera de la plataforma.

8 · Configuración y seguridad

- El archivo `.env` guarda los secretos: la clave del LLM (`GROQ_API_KEY`) y la cadena de conexión a la base de datos (`DATABASE_URL`). No se sube a internet (está en `.gitignore`). Hay una plantilla sin secretos, `.env.example`, para copiar y rellenar.
- La conexión a la base de datos se enlaza en `.env` (variable `DATABASE_URL`), la lee `config/settings.py` y la usa `integrations/db_backend.py`. Se abre en **modo solo lectura**.
- Los permisos (`config/permisos.py`) están fijados en `False` a prueba de todo: nunca escribir, nunca enviar, nunca crear, nunca aprobar. No son "un valor por defecto que se pueda cambiar": son una regla permanente.
- El servidor web (`servidor.py`) arranca con el "modo debug" apagado por defecto (activarlo expondría una consola peligrosa). Se puede encender solo para desarrollo con `FLASK_DEBUG=true`.

9 · Qué pasa cuando algo falla

| Situación                                               | Qué hace Lumen                                                                                                                                                                            |
|---------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| La IA (LLM) no está disponible o falla                  | Usa una <b>respuesta de reserva</b> automática, redactada por el código sin IA. Nunca se queda mudo.                                                                                      |
| Se agota la cuota diaria de la IA (error 429)           | Es el límite de tokens del plan gratuito de Groq (se resetea solo). Lumen sigue respondiendo con la versión de reserva. Se puede cambiar a un modelo con más cuota en <code>.env</code> . |
| La base de datos no responde                            | Devuelve un aviso claro ("no puedo confirmar este dato ahora") en vez de inventarse una respuesta.                                                                                        |
| El dato no existe                                       | Lo dice explícitamente ("no encuentro ese evento"), no lo aproxima.                                                                                                                       |
| Preguntas por <code>usuarios</code> o por una escritura | Bloqueo inmediato, marcado con nivel de riesgo, sin tocar la base de datos.                                                                                                               |

# 10 · Glosario completo

---

## Agente

Un programa que recibe una petición, hace un trabajo acotado y devuelve un resultado estructurado. Lumen es "el agente 04".

## API (Interfaz de Programación)

Un conjunto de "puertas" por las que un programa pide cosas a otro. La API de Lumen es `servidor.py`.

## Base de datos

Almacén ordenado de información en tablas. La nuestra es Postgres, alojada en Neon.

## Clave foránea (FK)

Un campo que "apunta" a una fila de otra tabla. Aquí tienen nombres como `id_cliente` o `id_sala`.

## Endpoint

Una dirección concreta dentro de una API. P. ej. `/chat` es el endpoint para preguntar.

## Fallback (reserva)

El plan B automático: si la IA falla, responde el código sin IA.

## Flask

La herramienta con la que está hecho el servidor web (`servidor.py`).

## Groq

El servicio que ejecuta el modelo de IA (Llama 3.3) al que Lumen le pide que redacte.

## JSON

Un formato de texto para intercambiar datos entre programas. La respuesta de Lumen viaja en JSON; tú solo ves el campo `resumen`.

## LLM (Modelo de Lenguaje)

La IA que entiende y redacta texto. Aquí solo redacta la respuesta; no consulta datos.

## Payload

El "paquete" de datos que se le pasa al agente con la pregunta y el contexto.

## Postgres / PostgreSQL

El tipo de base de datos que usamos.

## Neon

El proveedor en la nube que aloja esa base de datos Postgres.

## Solo lectura (read-only)

Modo en el que solo se puede consultar, nunca modificar. La conexión de Lumen es así.

## SQL

El lenguaje para preguntarle a la base de datos. Ej.: `SELECT * FROM eventos`. Lo escribe el código, no la IA.

## Token

La unidad con la que se mide el consumo de la IA (trozos de palabra). El plan gratuito tiene un tope diario.

## UUID

Un identificador único muy largo (tipo `8c4abda1-74a6-...`). Es el `id` real de cada fila; por eso la gente prefiere nombrar los eventos.

## .env

Archivo de configuración con los secretos. No se comparte ni se sube a internet.

---

Documento generado como material de apoyo del proyecto Lumen (Agente 04 · Copilot) — Ágora / Mitumi. Refleja el estado del código a 10/07/2026. Para el detalle técnico exacto, la fuente de verdad son los archivos del propio proyecto (`README.md`, `GUIA_EJECUCION.md` y el código en `src/`, `integrations/` y `config/`).